

## Licence d'Informatique L3

### Introduction aux Systèmes et Réseaux

#### Document technique n° 2 : La bibliothèque d'entrée-sortie Rio

On a vu en cours les opérations d'entrée-sortie sur les fichiers : `read()` et `write()`. On présente ici un ensemble d'opérations dites "robustes" qui utilisent `read()` et `write()` mais fournissent une interface plus commode et plus efficace. Elles sont utilisées pour les fichiers et les tubes, mais également pour les communications par *sockets* sur les réseaux. Ces opérations constituent la bibliothèque RIO<sup>1</sup> (*Robust Input-Output*). Le programme de ces opérations fait partie du fichier `csapp.c` disponible dans le placard ; voir aussi `csapp.h`.

N.B. L'objectif de cette présentation est double :

1. point de vue de l'utilisateur : présenter des fonctions plus sûres et plus commodes que les primitives élémentaires,
2. point de vue du concepteur : en examinant le détail de la programmation de ces fonctions, mieux comprendre le fonctionnement des entrées-sorties.

Tout le monde est intéressé par le premier aspect (ces fonctions seront réutilisées lors du TP sur les *sockets* et du projet de mini-serveur Web) ; selon votre intérêt vous pourrez regarder plus ou moins en détail la réalisation.

Il y a deux versions des opérations RIO : lecture et écriture non tamponnée (*unbuffered*) et lecture tamponnée (*buffered*).

Les opérations non tamponnées ont comme interface :

```
#include "csapp.h"
ssize_t rio_readn(int fd, void *usrbuf, size_t n);
ssize_t rio_writen(int fd, void *usrbuf, size_t n);
```

Elles transfèrent (au plus) `n` octets entre le fichier de descripteur `fd` et le tampon `usrbuf`. Elles renvoient le nombre exact d'octets lus et écrits, et `-1` en cas d'erreur. Noter que le type `ssize_t` est celui des entiers avec signe (pour inclure la valeur `-1` signalant une erreur) alors que le type `size_t` est celui des entiers positifs ou nuls.

Voici le programme de ces deux opérations :

```
ssize_t rio_readn(int fd, void *usrbuf, size_t n) {
    size_t nleft = n;
```

---

<sup>1</sup>La bibliothèque RIO a été développée par W. R. Stevens (*Unix Network Programming*, vol. 1, Prentice Hall, 1998). La version donnée ici est celle de R. E. Bryant et D. O'Hallaron (*Computer Systems : a Programmer's Perspective*, Prentice Hall, 2003).

```

ssize_t nread;
char *bufp = usrbuf;

while (nleft > 0) {
    if ((nread = read(fd, bufp, nleft)) < 0) {
        if (errno == EINTR) /* interrupted by sig handler return */
            nread = 0;      /* and call read() again */
        else
            return -1;      /* errno set by read() */
    }
    else if (nread == 0)
        break;              /* EOF */
    nleft -= nread;
    bufp += nread;
}
return (n - nleft);        /* return >= 0 */
}

ssize_t rio_writen(int fd, void *usrbuf, size_t n) {
    size_t nleft = n;
    ssize_t nwritten;
    char *bufp = usrbuf;

    while (nleft > 0) {
        if ((nwritten = write(fd, bufp, nleft)) <= 0) {
            if (errno == EINTR) /* interrupted by sig handler return */
                nwritten = 0;    /* and call write() again */
            else
                return -1;      /* errorno set by write() */
        }
        nleft -= nwritten;
        bufp += nwritten;
    }
    return n;
}

```

Examiner le fonctionnement de ces opérations. Noter que chacune des fonctions teste les cas d'erreur et redémarre l'entrée-sortie si celle-ci a été interrompue par un signal.

Les primitives de lecture tamponnée utilisent un tampon de lecture intermédiaire pour stocker des lignes de texte. Ce tampon est défini par le format suivant :

```

#define RIO_BUFSIZE 8192
typedef struct {
    int rio_fd;                /* descriptor for this internal buf */
    int rio_cnt;               /* unread bytes in internal buf */
    char *rio_bufptr;          /* next unread byte in internal buf */
    char rio_buf[RIO_BUFSIZE]; /* internal buffer */
} rio_t;

```

Le tampon est initialisé par la fonction ci-après.

```

void rio_readinitb(rio_t *rp, int fd) {
    rp->rio_fd = fd;
    rp->rio_cnt = 0;
    rp->rio_bufptr = rp->rio_buf;
}

```

Cette primitive associe un descripteur de fichier `fd` à un tampon de lecture de type `rio_t` situé à l'adresse `rp`. Elle doit être appelée une fois pour chaque descripteur ouvert.

Les primitives de lecture tamponnée ont pour interface :

```

ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n);
ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen);

```

La fonction `rio_readnb(rio_t *rp, void *usrbuf, size_t n)` lit au plus `n` octets depuis un fichier (associé au tampon `rp`) vers un tampon destinataire `usrbuf`.

La fonction `rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen)` lit une ligne de texte (y compris le caractère de fin de ligne) depuis un fichier (associé au tampon `rp`) vers un tampon destinataire `usrbuf`. Un caractère NULL est placé en fin du texte. La fonction lit au plus `maxlen - 1` caractères (si la ligne lue est plus longue, elle est tronquée à `maxlen - 1`, et toujours terminée par NULL).

Le programme des fonctions de lecture tamponnée est donné ci-après. Ces fonctions utilisent une primitive commune de lecture `rio_read` (dont le programme figure à la fin), qui a la même sémantique que la primitive `read`, mais qui lit dans le tampon et réalimente celui-ci à partir du fichier quand il est vide, en appelant `read`.

```

/* lecture tamponnée d'une ligne */
ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen) {
    int n, rc;
    char c, *bufp = usrbuf;

    for (n = 1; n < maxlen; n++) {
        if ((rc = rio_read(rp, &c, 1)) == 1) {
            *bufp++ = c;
            if (c == '\n')
                break;
        } else if (rc == 0) {
            if (n == 1)
                return 0; /* EOF, no data read */
            else
                break; /* EOF, some data was read */
        } else
            return -1; /* error */
    }
    *bufp = 0;
    return n;
}

/* lecture tamponnée d'une suite d'octets */
ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n) {
    size_t nleft = n;
    ssize_t nread;

```

```

char *bufp = usrbuf;
while (nleft > 0) {
    if ((nread = rio_read(rp, bufp, nleft)) < 0) {
        if (errno == EINTR) /* interrupted by sig handler return */
            nread = 0;      /* call read() again */
        else
            return -1;      /* errno set by read() */
    }
    else if (nread == 0)
        break;              /* EOF */
    nleft -= nread;
    bufp += nread;
}
return (n - nleft);        /* return >= 0 */
}

/*
 * cette fonction est une enveloppe (wrapper) pour la primitive read.
 * Elle transfère min(n, rio_cnt) octets depuis un tampon interne vers
 * un tampon chez l'utilisateur. n est le nombre d'octets dont le
 * transfert est demandé, et rio_cnt le nombre d'octets non lus dans
 * le tampon interne. rio_read remplit le tampon (en utilisant read) si
 * elle trouve le tampon vide.
 */
static ssize_t rio_read(rio_t *rp, char *usrbuf, size_t n) {
    int cnt;

    while (rp->rio_cnt <= 0) { /* refill if buf is empty */
        rp->rio_cnt = read(rp->rio_fd, rp->rio_buf,
                           sizeof(rp->rio_buf));
        if (rp->rio_cnt < 0) {
            if (errno != EINTR) /* interrupted by sig handler return */
                return -1;
        }
        else if (rp->rio_cnt == 0) /* EOF */
            return 0;
        else
            rp->rio_bufptr = rp->rio_buf; /* reset buffer ptr */
    }

    /* Copy min(n, rp->rio_cnt) bytes from internal buf to user buf */
    cnt = n;
    if (rp->rio_cnt < n)
        cnt = rp->rio_cnt;
    memcpy(usrbuf, rp->rio_bufptr, cnt);
    rp->rio_bufptr += cnt;
    rp->rio_cnt -= cnt;
    return cnt;
}

```

Examiner ces fonctions et comprendre leur mode de fonctionnement. Ensuite, programmer la recopie d'un fichier de texte (constitué d'une suite de lignes) depuis l'entrée standard vers la sortie standard, en utilisant la lecture tamponnée et l'écriture.